Image Processing from Scratch

Computer Vision — Task 1 Report

Farah Yehya Abdelazim, Yousef Samy Abdelgawed, Kareem Mohamed Elsayed

October 19, 2026

1 Overview

We built a web app that applies classic image-processing operations to an uploaded image and shows the result next to the original. The math is written from scratch with NumPy. OpenCV is used only for file reading, random noise generation, Canny and the DFT. The backend is FastAPI (one endpoint per operation) and the interface is React.

Implemented: noise, low-pass filters, edge detection, histograms (PDF/CDF), equalization, normalization, RGB/gray, frequency filters and hybrid images.

How the numbers below were measured: we ran our own `services/` code on a 512×512 grayscale test image (cameraman). PSNR is measured against the clean image; higher is better. The screenshots use other images (Lena, a cat photo).

2 User Interface

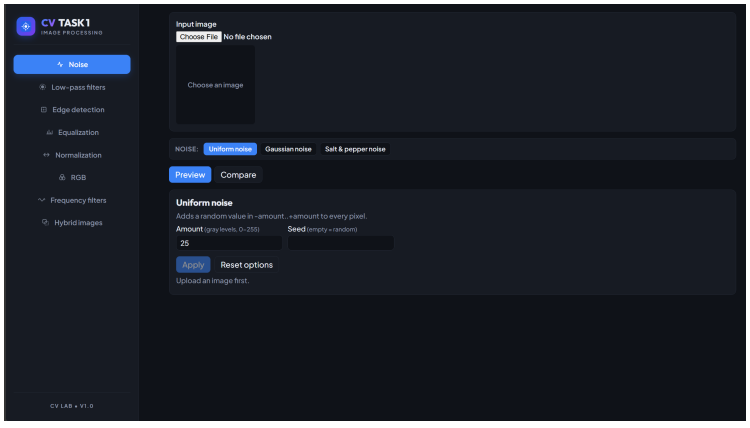


Figure 1: Overall interface: sidebar, image input, tabs and result area.

- **Sidebar:** collapsible task groups; the options form is generated from one list (`tasks.js`), with units for every option.
- **Apply / Reset / Use result as input:** the last button feeds the result back as the new input, so steps can be chained (add noise, then filter it).
- **Before / After:** side by side or with a drag slider; a badge shows $\Delta\mu$ and $\Delta\sigma$.
- **Live histogram** under every image: gray / RGB / one channel, optional PDF and CDF, and *Overlay Before*.

- **Compare tab:** runs several variants (noise types, filters, edge detectors, gray vs. color equalization) on the same image, each with its own options.
- **Readable errors:** an even kernel size shows "kernel_size must be a positive odd integer" (HTTP 400).



(a) Slider mode



(b) Histogram with PDF/CDF and overlay

Figure 2: Before/After slider and histogram view.

3 Observations per Task

Noise

Parameter changed	What we observed
Uniform *amount* 10 → 25 → 50	PSNR 32.9 → 25.1 → 19.3 dB. Grain grows and the histogram widens. Extreme values clip at 0 and 255.
Gaussian *sigma* 10 → 20 → 40	PSNR 28.2 → 22.4 → 16.9 dB. Same trend. Uniform noise of amount a has std $a/\sqrt{3}$, so it is milder than Gaussian with $\sigma = a$.
Gaussian *mean* ±30	The whole image gets brighter or darker (mean shifts by about ±29, less than 30 because of clipping). No extra grain.
Salt & pepper *amount* 0.02 → 0.10	PSNR 21.8 → 14.7 dB. At 0.10, about 5% of pixels turn white and 5% black. The histogram gets two spikes at 0 and 255; the rest is unchanged.
salt_vs_pepper 0.5 → 0.9	White dots dominate: 9.1% white vs. 1% black (amount 0.10).
seed	Same seed gives the same noise, so results are reproducible.



(a) Gaussian noise (σ=30)



(b) Salt & pepper noise

Figure 3: Two noise types on the same image.

Low-pass filters

PSNR after filtering (noisy input: 22.4 dB for Gaussian noise $\sigma = 20$, 17.8 dB for 5% salt & pepper):

	Gaussian noise			Salt & pepper		
Kernel	Average	Gaussian	Median	Average	Gaussian	Median
3×3	27.4	**28.0**	27.0	24.9	24.7	**30.1**
5×5	26.2	**27.8**	26.8	25.1	25.7	**27.8**
9×9	23.8	**26.3**	24.6	23.4	**25.4**	24.8

- **Kernel size:** larger kernels blur edges more. On clean data the average filter drops from 29.4 dB (3×3) to 24.0 (9×9) to 22.2 (15×15).
- **Gaussian vs. average:** at the same size the Gaussian filter is better, because it keeps the center pixel more.
- **Gaussian *sigma*** (15×15 kernel): $\sigma = 0.5$ barely changes the image (37.6 dB vs. original); $\sigma = 8$ is almost identical to the average filter (47 dB between the two).
- **Salt & pepper:** average and Gaussian smear the dots into blobs. The median removes them and keeps edges sharper, but a large window (9×9) loses detail and thin lines.
- **Cost:** our median 5×5 takes 0.10 s on 512×512 vs. 0.003 s for `cv2.medianBlur` (about 40× slower).



(a) Average filter



(b) Median filter

Figure 4: Average vs. median filter on a salt & pepper image.

Edge detection

What we changed	**What we observed**
Method	Gradient X responds to vertical edges, Gradient Y to horizontal ones. Roberts measures the two diagonals, so a vertical edge shows in both of its outputs.
Add Gaussian noise ($\sigma = 20$) before the detector	Mean edge magnitude: Sobel 49 → 108 (2.2×), Prewitt 36 → 77 (2.1×), Roberts 13 → 39 (3.0×). Roberts uses only 2 pixels, so it is the most noise-sensitive.
Edge response / noise response (synthetic step edge)	Prewitt 9.9, Sobel 9.3, Roberts 5.7.
Gaussian 5×5 before Sobel	Noise response drops by about 70% (86.3 → 26.0). On the noisy image the mean magnitude falls back to 45, close to the clean value.

Canny: number of edge pixels (cameraman, Gaussian noise $\sigma = 20$ for the last two columns):

Thresholds (low / high)	Clean	Noisy	Blur 5×5, then Canny
50 / 150	30 980	91 871	12 210
100 / 200	19 686	70 594	6 981
150 / 300	9 452	25 148	5 174

- **Higher thresholds** remove noise edges (noisy: 91 871 → 25 148) but also remove weak real edges (clean: 30 980 → 9 452).
- **Blurring first** gives the biggest reduction on noisy images, since OpenCV's Canny does not smooth. It also removes some weak real edges.
- A synthetic circle image shows the same trend (1011 edge pixels at 50/150 vs. 449 at 100/200).

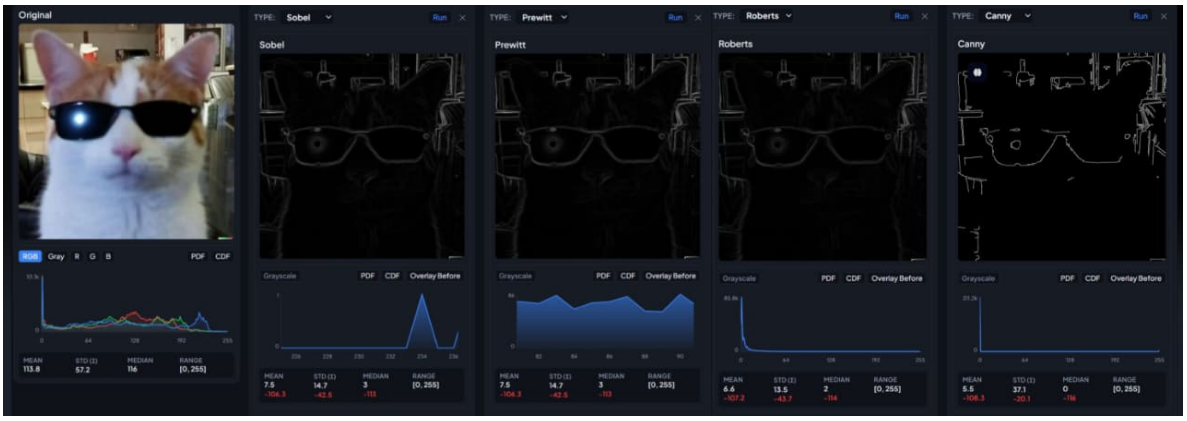


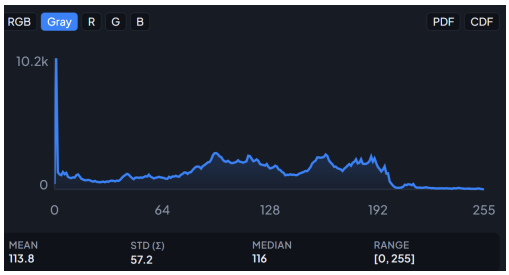
Figure 5: Compare tab: Sobel, Prewitt, Roberts and Canny on one image (a different test image).

Histogram, equalization and normalization

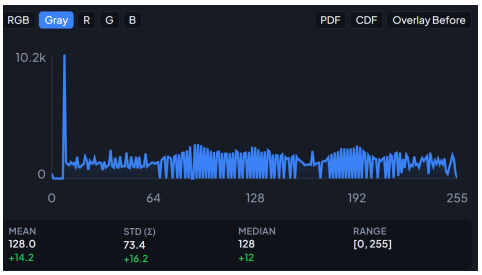
Test image: the cameraman squeezed to low contrast (range 80–156, $\sigma = 22.1$).

What we changed	**What we observed**
Normalize to [0, 255]	σ rises 22.1 → 74.2. The histogram keeps its shape and is only stretched.
Normalize to a narrow range [50, 150]	σ only 29.1, so contrast stays low.
One extreme pixel (255) added	Normalization can stretch only up to 32.2 σ, because the max is now 255. Equalization is not affected (74.0 with and without it).
Equalization	σ 22.1 → 74.0. The histogram spreads over 0–255 and the CDF becomes almost a straight line.
Equalization on a color image	Done on the luminance (Y of YCbCr) only, so hues are preserved.

- On this image normalization and equalization give almost the same σ. The difference is the shape: equalization flattens the histogram, normalization does not.
- Equalization can amplify noise in flat regions.



(a) Original



(b) Equalized

Figure 6: Histogram before and after equalization (mean 113.8 → 128.0, σ 57.2 → 73.4).

RGB and gray

- Gray $= 0.299R + 0.587G + 0.114B$. Green has the biggest weight because the eye is most sensitive to it. The weights sum to 1, so white stays 255.
- The R, G, B histograms (with CDF) show which channel dominates a color image.
- Our gray image differs from `cv2.cvtColor` by at most 1 level (on 0.12% of pixels, rounding).

Frequency filters and hybrid images

Ideal masks: a circle of radius *cutoff* (in frequency samples from the centered spectrum). Cameraman:

Cutoff	5	15	30	60	120
Low-pass PSNR vs. original (dB)	17.9	20.8	22.8	25.8	29.6
High-pass std of result	32.5	23.2	18.4	13.1	8.4

- **Low-pass:** a small cutoff gives a very blurry image; a large cutoff approaches the original.
- **Ringing:** the ideal mask creates ripples around edges. At cutoff 30 the output range is -29 to 269, outside 0–255. A Gaussian mask at cutoff 15 stays inside (4 to 227).
- **High-pass:** a larger cutoff keeps less (only edges and fine texture); flat regions turn uniform gray.
- **Hybrid** (Gaussian masks, so no ringing): a smaller *cutoff_low* keeps only the coarse shape of the far image; a larger *cutoff_high* keeps only the finest details of the near image. Close up we see the near image; far away (or shrunk) the far image.



(a) Low-pass, small cutoff



(b) Hybrid image

Figure 7: Ideal low-pass with a small cutoff, and a hybrid image.

4 Implementation Notes

- **Padding:** copying the edge pixel avoids the fake bright frame that zero padding creates. Our border pixels therefore differ from `cv2.Sobel` (reflect padding).
- **Gradient range:** for a 0→255 step, the maximum response is Sobel 1020, Prewitt 765, Roberts 255. After normalization all reach 255, so brightness cannot be compared across methods.
- **Display of float results:** gradients are min-max normalized. Plain `astype(uint8)` would wrap 760 around to 248.
- **Histogram speed:** `np.bincount` takes 4.2 ms vs. 411.9 ms for a Python loop on 1000×1000 (about 99× faster).
- **FFT cache:** the DFT does not depend on the cutoff, so FFTs are cached (SHA-256 of the upload, 4 entries). Changing only the cutoff is fast.
- **Checks:** Sobel is identical to `cv2.Sobel` on inner pixels; histogram counts equal `np.histogram`; all endpoints were tested through the frontend proxy.

5 Conclusion

- Choose the filter by the noise: Gaussian/average for Gaussian-like noise, median for salt & pepper with a small window.
- Larger kernels and lower cutoffs generally trade noise reduction for detail.
- Smoothing before edge detection gives the biggest improvement; Roberts is the most noise-sensitive detector.
- Normalization and equalization raised contrast equally on our test image. Equalization is robust to outliers; normalization is not.
- **Limitations:** edges are computed on gray (two colors of equal brightness give no edge); Task 9 uses ideal masks that ring; Canny and the DFT come from OpenCV; our median filter is much slower than OpenCV's.